DHBW Heilbronn

Software Engineering II.1 – Block 4

Implementierung und Versionsmanagement

Zusammenarbeit, Build und KI nachvollziehbar organisieren

Wo stehen wir im Kurs?

Block 1	Block 2	Block 3	**Block 4**	Block 5	Block 6	Block 7
4 UE	4 UE	4 UE		8 UE		4 UE
Einführung & Vorgehensmodelle	Requirements Engineering	Entwurf & Entwurfsmuster	Implementierung & Versionsmanagement	Testen (Blackbox)	Whitebox, Wartung & Change Request	Management & Abschlusspräsentation

Durchgängiges Gruppenprojekt „Photo Album“: jeder Block liefert ein prüfbares Artefakt ins Repository

Teams, Repo, Vorgehen	REQUIREMENTS.md Use Cases · AK · Prototyp	ARCHITECTURE.md Klassen- · Sequenzdiagramm	Build · Git · PR Review · CI grün	TESTING.md Unit/Integr. · Blackbox	Coverage · Refactoring Change Request	RISK_ANALYSIS.md Präsentation

Heute wird der Entwurf ausführbar

- Reproduzierbarer Build mit Maven oder Gradle; Dependencies bewusst verwalten
- Debugging als Hypothesentest, nicht als Raten
- Git als Teamprotokoll: Branches, kleine Commits, Pull Requests, Konflikte fachlich lösen
- Code Review als Quality Gate – und als Lernwerkzeug
- CI-Pipeline, die Build und Tests bei jedem Push ausführt; Releases und Versionen
- KI beim Programmieren: Coding Agents, Repository Rules, Versionierung von Prompts und Skills

Projektergebnis dieses Blocks

Lauffähiger Kern: Album erstellen + anzeigen (UI → UseCase → Domäne → Repository)

Build läuft auf jedem Rechner mit einem Befehl

Mindestens ein PR mit dokumentiertem Review

CI-Pipeline grün (Build + Tests)

AI_USAGE.md Fall 3: Coding-Agent-Einsatz bewertet

Einstieg Übung: Repository Forensics – was erzählt ein Repository über ein Team?

```
# Repository A                          # Repository B
git log --oneline                       git log --oneline
a91f3c2 fix                             7c2e1aa feat(album): reject duplicate
photo (FA-05)
b02d7e1 fix again                       3b9d0f4 test(album): boundary
99/100/101 photos
c7a8810 final version                   e41c77d refactor(usecase): extract
ImageHasher port
d19bb04 final version 2                 9a1f2b6 docs(adr): ADR-001 repository
interface
e33c0ff Merge branch 'main' of ...      5d3e8c1 feat(ui): create album dialog
(#12)
f4400aa upload everything               2f7a9e0 ci: add GitHub Actions build
(#9)
0a1b2c3 Initial commit                  0a1b2c3 chore: maven skeleton, package
layout

Contributors: 1 (von 4)                 Contributors: 4 – PRs: 11 – Reviews: 9
```

- 5 Minuten, Team: Notieren Sie drei Spuren guter Zusammenarbeit und zwei Warnsignale.
- Was können Sie über Prozess, Tests und Verantwortung ablesen – ohne eine Zeile Code zu sehen?
- Welches Repository würden Sie als Prüferin lieber bewerten? Warum?

1

Repository, Build und Dependencies

Ein Befehl, jeder Rechner, gleiches Ergebnis.

Eine kleine Repository-Struktur genügt

```
photo-album/
├── README.md               # Start: Zweck, Build, Run, Team
├── REQUIREMENTS.md         # Block 2
├── ARCHITECTURE.md         # Block 3
├── TESTING.md              # Block 5
├── AI_USAGE.md             # laufend
├── RISK_ANALYSIS.md        # Block 7
├── AGENTS.md               # Regeln für Coding Agents (heute)
├── pom.xml | build.gradle.kts
├── .github/workflows/ci.yml
├── docs/
│   ├── adr/ADR-001-repository.md
│   ├── design/*.puml | *.png
│   └── prototype/
├── src/main/java/de/dhbw/photoalbum/
│   ├── domain/         Album, Photo,Exceptions
│   ├── application/    AddPhotoUseCase, ports (AlbumRepository, ImageHasher)
│   ├── infrastructure/ FileAlbumRepository, Sha256ImageHasher
│   └── ui/             AlbumWindow (JavaFX) oder ConsoleUi
└── src/test/java/de/dhbw/photoalbum/   (Spiegel von main)
```

- Pakete spiegeln die Schichten aus Block 3 – Struktur ist Architektur, die man sieht
- Dokumente auf oberster Ebene, so wie die Projektbeschreibung sie verlangt
- src/test spiegelt src/main: Test zu Album liegt in domain/AlbumTest
- Kein Build-Output, keine IDE-Dateien im Repository: .gitignore für target/, build/, .idea/, *.iml
- Bewertung: „Repository sauber und nachvollziehbar strukturiert“ – 1 Punkt, aber Voraussetzung für alle anderen

Maven oder Gradle? Beides ist richtig – entscheiden Sie einmal

Maven

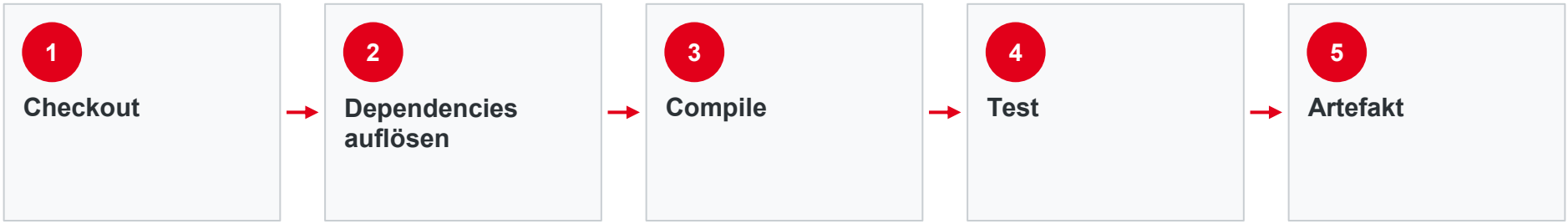
- Deklaratives **pom.xml**, feste Konventionen (src/main/java, src/test/java)
- Lebenszyklus: validate → compile → test → package → verify → install
- Größtes Java-Ökosystem an Plugins und Dokumentation; jede IDE versteht es
- Empfehlung für dieses Projekt, wenn niemand im Team Gradle-Erfahrung hat

Gradle (Kotlin DSL)

- Build-Skript in Kotlin/Groovy, flexibler, kürzer
- Inkrementelle Builds und Build-Cache – schneller bei großen Projekten
- Mehr Gestaltungsspielraum (= mehr Möglichkeiten, es kompliziert zu machen)
- Sinnvoll, wenn das Team es kennt oder Android-Erfahrung hat

Entscheidung als ADR-00x dokumentieren (zwei Sätze reichen). Beide erfüllen alle Projektanforderungen: JUnit 5, JaCoCo, CI.

Der reproduzierbare Build: fünf Schritte, ein Befehl



1 Checkout	2 Dependencies auflösen	3 Compile	4 Test	5 Artefakt
git clone – nichts, was nicht im Repository ist, darf nötig sein	aus pom.xml/build.gradle mit festen Versionen; Wrapper legt Build-Tool-Version fest	Java 21, festgelegt in der Build-Datei, nicht in der IDE	JUnit 5 läuft automatisch; ein roter Test bricht den Build	jar in target/ (Maven) bzw. build/libs/ (Gradle) – das Release-Kandidat

Test: Ein Teammitglied klont das Repository in ein leeres Verzeichnis und führt ./mvnw verify aus. Grün? Dann ist der Build reproduzierbar. Das ist Übung 1.

Minimale pom.xml für das Photo Album

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>de.dhbw</groupId>
  <artifactId>photo-album</artifactId>
  <version>0.1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <junit.version>5.11.4</junit.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>${junit.version}</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build><plugins>
    <plugin><groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId><version>3.5.2</version></plugin>
    <plugin><groupId>org.jacoco</groupId><artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.12</version>
      <executions><execution><goals><goal>prepare-agent</goal></goals></execution>

<execution><id>report</id><phase>verify</phase><goals><goal>report</goal></goals></exe
cution>
      </executions></plugin>
  </plugins></build>
</project>
```

- Feste Versionen – keine RANGES, kein LATEST
- release 21 statt source/target: eine Zeile, eindeutig
- JUnit 5 nur im Test-Scope: landet nicht im Artefakt
- Surefire führt Tests aus, JaCoCo misst Coverage (Block 5/6)
- SNAPSHOT bis zum ersten Release-Tag v0.1.0

Dependency Management ist eine Entwurfsentscheidung

- Jede Bibliothek ist Kopplung an fremden Code mit eigenem Lebenszyklus, eigenen Lücken und eigener Lizenz
- Direkt vs. transitiv: Sie deklarieren 3 Dependencies, im Klassenpfad landen 40 – `mvn dependency:tree` zeigt sie
- Versionen explizit festlegen (Reproduzierbarkeit); Updates bewusst: Release Notes lesen, in eigenem Branch, Tests laufen lassen
- Automatisierte Hinweise nutzen: GitHub Dependabot meldet verwundbare Versionen – bewerten, nicht blind mergen
- Nicht benötigte Bibliotheken entfernen: weniger Angriffsfläche, schnellerer Build
- Wiederverwendung (17_Reuse):
  - Nutzen = gesparter Aufwand − Integrations- und Wartungskosten;
  - für ein Projekt dieser Größe reichen JUnit, JaCoCo und ggf. JavaFX

Photo Album: Jede Dependency braucht einen Grund im ADR.

Übung: Build-Check

Übung

Reproduzierbarkeit testen, bevor die CI es tut.

30 min

Team

AUFTRAG

1. Gerüst anlegen. Jedes Teammitglied, klont das Repository in ein neues Verzeichnis (nicht das bestehende!).
2. ./mvnw verify bzw. ./gradlew build ausführen. Grün?
3. Wenn rot: Ursache notieren (fehlender Wrapper, falsche Java-Version, IDE-Abhängigkeit, absoluter Pfad) und beheben – als Issue + Commit.

ERGEBNIS Build grün aus frischem Clone; ggf. Fix-Commit „build: …“.

DANACH IM PLENUM Handzeichen: Welche Teams waren beim ersten Versuch grün? Welche Ursachen gab es?

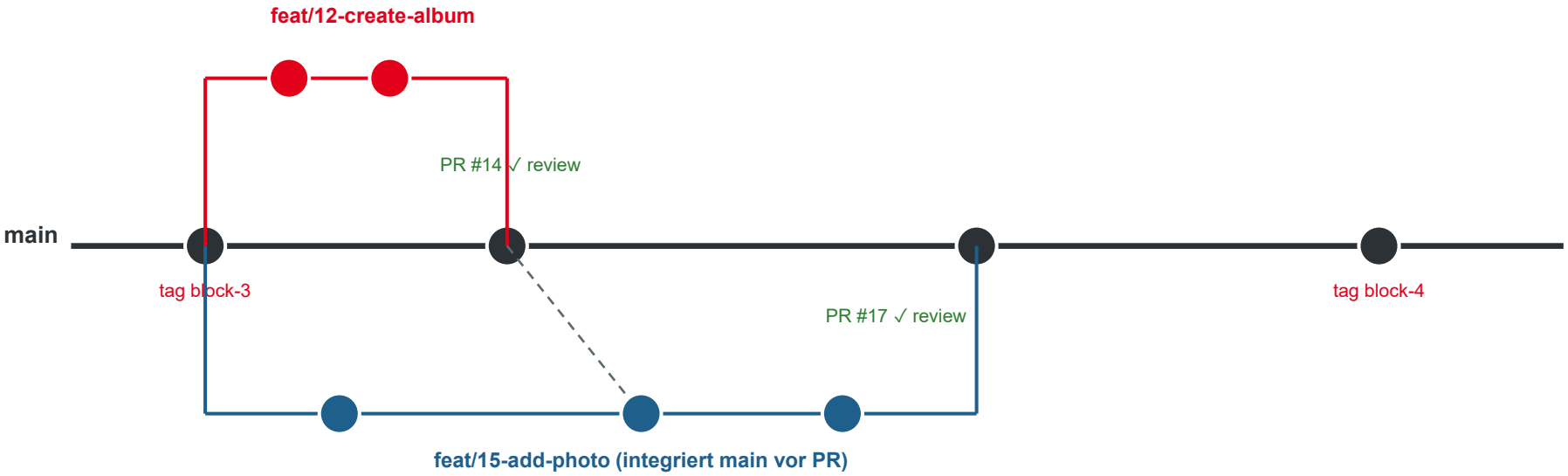
2

Git-Workflow

Branches, kleine Commits, Pull Requests – und Konflikte als Feedback.

Ein schlanker Git-Workflow für vier Personen

main ist geschützt (Branch Protection: PR + 1 Review + grüne CI). Kurze Branches, kleine PRs, schnelle Reviews.



1. Issue anlegen (Was? Warum? Akzeptanzkriterium)	2. Branch: feat/<issue>-<kurz>	3. kleine Commits, jeder baut grün	4. Pull Request: Ziel, Prüfung, Risiken	5. Review durch andere Person	6. CI grün → Merge → Branch löschen

Commit und Pull Request beantworten verschiedene Fragen

Commit: eine atomare Änderung

- Betreff ≤ 72 Zeichen, Imperativ, Typ voran: feat | fix | test | refactor | docs | ci | chore
- Bezug: Anforderungs-ID oder Issue: „feat(album): reject duplicate photo (FA-05, #15)“
- Body (optional): Warum, nicht Was – das Was steht im Diff
- Jeder Commit baut grün – sonst ist git bisect wertlos
- Kein „WIP“, kein „fix“, kein „final“

Pull Request: eine fachliche Einheit

- Titel wie ein Commit; Beschreibung: Ziel, Umsetzung in 3 Sätzen, wie geprüft (Tests, manuell), offene Risiken
- Klein: < 400 geänderte Zeilen – sonst reviewt niemand wirklich
- Verlinkt Issue (closes #15) und ggf. ADR
- Checkliste im PR-Template: Tests ergänzt? Doku aktualisiert? AI_USAGE, falls KI beteiligt?
- Review-Kommentare werden beantwortet oder umgesetzt, nie ignoriert

Commit und Pull Request beantworten verschiedene Fragen

Commit: eine atomare Änderung

- Betreff ≤ 72 Zeichen, Imperativ, Typ voran: feat | fix | test | refactor | docs | ci | chore
- Bezug: Anforderungs-ID oder Issue: „feat(album): reject duplicate photo (FA-05, #15)“
- Body (optional): Warum, nicht Was – das Was steht im Diff
- Jeder Commit baut grün – sonst ist git bisect wertlos
- Kein „WIP“, kein „fix“, kein „final“

Pull Request: eine fachliche Einheit

- Titel wie ein Commit; Beschreibung: Ziel, Umsetzung in 3 Sätzen, wie geprüft (Tests, manuell), offene Risiken
- Klein: < 400 geänderte Zeilen – sonst reviewt niemand wirklich
- Verlinkt Issue (closes #15) und ggf. ADR
- Checkliste im PR-Template: Tests ergänzt? Doku aktualisiert? AI_USAGE, falls KI beteiligt?
- Review-Kommentare werden beantwortet oder umgesetzt, nie ignoriert

Konflikte sind Integrationsfeedback, kein Unfall

```
<<<<<<< HEAD (main)
    if (photos.size() >= MAX_PHOTOS)
        throw new AlbumFullException(name, MAX_PHOTOS);
=======
    if (photos.size() >= maxPhotos)          // aus feat/20-configurable-
limit
        throw new AlbumFullException(name, maxPhotos);
>>>>>>> feat/20-configurable-limit

# Vorgehen:
git switch feat/20-configurable-limit
git merge main            # oder: git rebase main
# Konflikt fachlich lösen: Welche Regel gilt? (nicht: "beide behalten")
./mvnw verify             # Build UND Tests nach der Auflösung
git add Album.java && git commit
```

- Ein Konflikt heißt: zwei Personen haben dieselbe Entscheidung unterschiedlich getroffen – das ist eine Fachfrage, keine Textfrage
- Nie mechanisch „accept both“ – erst verstehen, was beide wollten, dann ggf. absprechen
- Kleine Branches, häufig main integrieren → Konfliktfläche klein
- Nach jeder Auflösung: Build und Tests – Konfliktauflösung ist die häufigste Quelle stiller Fehler
- IDE-Merge-Tool nutzen (3-Wege-Ansicht: Basis, meins, ihres)

4

Code Review und Continuous Integration

Zwei Quality Gates: ein menschliches, ein automatisches.

Code Review prüft mehr als Stil – in dieser Reihenfolge

Ebene	Leitfrage der Reviewerin	Beispiel im Photo Album
1 Fachliche Korrektheit	Erfüllt der Code die Anforderung/das Akzeptanzkriterium? Grenzfälle?	Wird bei genau 100 Fotos abgelehnt oder bei 101? (FA-06, AK-06.2)
2 Verantwortlichkeiten	Liegt die Logik in der richtigen Schicht? Kopplung?	Prüft die UI die Namenslänge? → gehört in Album
3 Tests	Gibt es Tests für das Risiko? Testen sie Verhalten oder Implementierung?	Test für Duplikat vorhanden, aber nur mit gleichem Dateinamen
4 Fehlerbehandlung & Sicherheit	Was passiert bei null, leer, zu groß, nicht lesbar?	readAllBytes auf 20-MB-Datei ohne Größenprüfung
5 Verständlichkeit	Versteht eine Person ohne Kontext in 2 Minuten, was passiert?	Methode processStuff() mit 60 Zeilen
6 Stil	Konventionen, Formatierung – automatisierbar, zuletzt	Einrückung, Importe – Formatter/Checkstyle statt Mensch

Regel für Review-Kommentare: Frage statt Befehl, Bezug auf Anforderung oder Prinzip, Vorschlag beilegen. „Warum …?“ ist erlaubt, „falsch“ nicht.

Review-Kommentare: so nicht – so ja

WENIG HILFREICH

„Das ist falsch.“ · „Warum machst du das so?“ · „Bitte umschreiben.“ · „LGTM“ (ohne den Diff geöffnet zu haben) · 14 Kommentare zu Leerzeichen, keiner zur Logik.

HILFREICH: BEZUG, FRAGE, VORSCHLAG

„Bei genau 100 Fotos lässt `>` das 101. Foto noch zu – AK-06.2 verlangt Ablehnung ab dem 101. Meinst du `>=`? Ein Test mit 100/101 würde das absichern.“

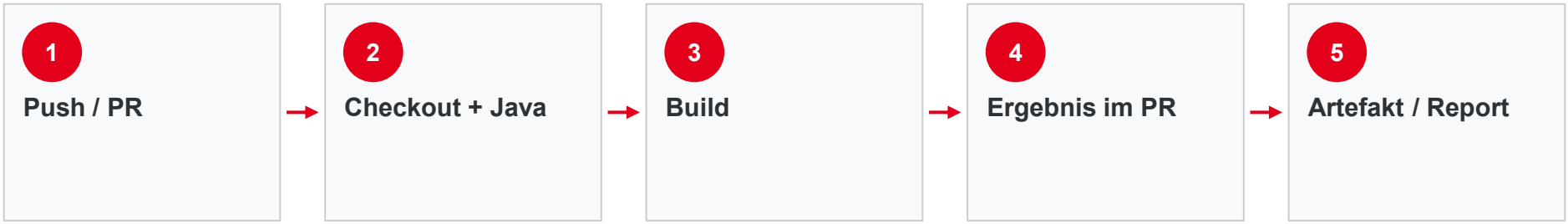
„Die Namensprüfung liegt hier in der UI; nach ADR-002 gehört sie in Album. Soll ich ein Issue dafür anlegen, oder passt es in diesen PR?“

Ein guter Kommentar nennt den Ort, die Regel (Anforderung/ADR/Prinzip), die Konsequenz und einen Vorschlag – und lässt den Autoren die Entscheidung.

```
if (photos.size() > MAX_PHOTOS)  throw ...   // 100 > 100 → false → KEINE Ablehnung
photos.add(photo);                           // jetzt sind es 101 Fotos ×

if (photos.size() >= MAX_PHOTOS) throw ...   // 100 >= 100 → true → Ablehnung ✓
```

CI als automatisches Quality Gate



jede Änderung auf jedem Branch löst den Lauf aus

frische Maschine, festgelegte Java-Version – der Reproduzierbarkeits-Test

./mvnw verify – kompiliert, führt Tests aus, erzeugt Coverage

grün/rot direkt am Pull Request; rot blockiert den Merge

jar und JaCoCo-Report als Artefakt herunterladbar

CI beantwortet die Frage „funktioniert es woanders?“ jedes Mal, automatisch, in zwei Minuten. Das ist die Grundlage für Regressionstests (Block 6) und Quality Gates (Block 7).

Minimale CI-Pipeline: .github/workflows/ci.yml

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - name: Build and test
        run: ./mvnw --batch-mode verify
      - name: Upload coverage report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

- Läuft bei Push auf main und bei jedem PR
- setup-java legt die Version fest – dieselbe wie in pom.xml
- cache: maven spart Minuten pro Lauf
- verify statt test: JaCoCo-Report entsteht in der verify-Phase
- Branch Protection: „Require status checks to pass“ auf den Job build
- Gradle: ./gradlew build; Report unter build/reports/jacoco/

DevOps und DevSecOps: Kultur hinter der Pipeline

DevOps

- Entwicklung und Betrieb teilen Verantwortung für das laufende System („you build it, you run it“)
- Kurze Feedbackschleifen: CI, automatisierte Lieferung (CD), Monitoring
- Kleine, häufige Änderungen statt großer Releases – Risiko pro Änderung sinkt
- Messbar über DORA-Metriken (Block 7): Lead Time, Deployment Frequency, Change Failure Rate, Time to Restore

DevSecOps

- Sicherheit wird in die Pipeline gezogen, statt am Ende geprüft („shift left“)
- Dependency-Scanning (Dependabot, OWASP Dependency-Check) bei jedem Build
- Secrets nie im Repository: .env in .gitignore, GitHub Secrets für CI
- Statische Analyse (SpotBugs, Checkstyle) als weiterer Job – Nachweise automatisch

Für das Projekt: Dependabot aktivieren (eine Datei), Secrets-Regel im README, optional ein SpotBugs-Job. Mehr ist nicht nötig.

Release Management: ein bekannter, reproduzierbarer Stand

- **Version identifiziert einen Stand: Semantic Versioning MAJOR.MINOR.PATCH**
  - 0.1.0 erster lauffähiger Kern · 0.2.0 Fotos · 1.0.0 Pflichtfunktionalität vollständig · 1.1.0 Change Request
- **Tag verbindet Quellcode und Release: git tag -a v0.1.0 -m "…" && git push --tags**
  - Blockabgaben: block-2 … block-7 als Tags – so sehe ich, was wann fertig war
- **Changelog erklärt relevante Änderungen für Menschen (CHANGELOG.md, „Keep a Changelog“)**
- **Release Candidate (1.0.0-rc1) erlaubt gezielte Prüfung vor der Freigabe**
- **Hotfix: Branch vom Tag, Fix, Test, neuer PATCH-Tag – nachvollziehbar, nicht „auf dem Server geändert“**
- **Bekannte Einschränkungen ins Release notieren – Ehrlichkeit ist ein Qualitätsmerkmal (Block 7: Release-Entscheidung)**

5

KI-Perspektive

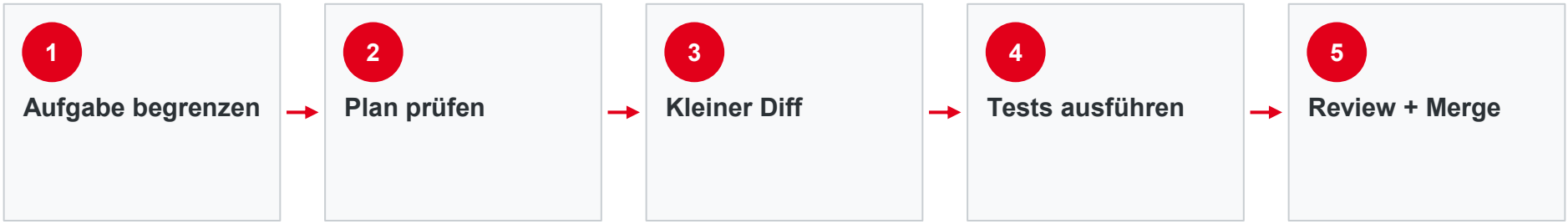
Coding Agents, Repository Rules und Versionierung von Prompts, Skills und Modellen.

KI beim Programmieren bleibt überprüfungspflichtig

- Assistent (Autocomplete, Chat) schlägt vor – Agent (Copilot Agent, Claude Code, Cursor) ändert Dateien, führt Builds und Tests aus, öffnet PRs
- Kontext und Grenzen explizit geben: Architektur, Konventionen, verbotene Bereiche – sonst rät der Agent
- Kleine Änderungspakete verlangen: ein Issue, ein Branch, ein PR – wie bei Menschen
- Jede KI-Änderung wird gelesen wie ein PR eines neuen Teammitglieds: fachlich, Tests, Sicherheit
- Risiken: erfundene APIs, Lizenzverstöße durch kopierten Code, Secrets im Prompt, subtil falsche Tests, die immer grün sind
- Nutzen im Projekt: Boilerplate, Testfälle aus Akzeptanzkriterien, Refactoring-Vorschläge, Dokumentation – dokumentiert in AI_USAGE.md

Bewertungskriterium: „KI-Einsatz transparent und kritisch dokumentiert“ (3 Punkte) – die Bewertung des Ergebnisses zählt, nicht die Nutzung.

Sicherer Workflow mit Coding Agents



Issue mit Akzeptanzkriterium; Scope: eine Klasse, ein Use Case

Agent zuerst planen lassen; Plan gegen Architektur/ADRs lesen

Änderung auf eigenem Branch; Agent darf nicht auf main

Agent führt ./mvnw verify aus; neue Tests werden gelesen, nicht nur gezählt

Mensch reviewt wie jeden PR; AI_USAGE.md: übernommen / angepasst / verworfen

Repository Rules steuern Coding Agents: AGENTS.md

```
# AGENTS.md – Regeln für KI-Coding-Agents in diesem Repository

## Build & Test
- Build: ./mvnw verify   (muss grün sein, bevor ein PR geöffnet wird)
- Tests: JUnit 5 in src/test/java, Spiegel von src/main/java

## Architektur (siehe ARCHITECTURE.md, docs/adr/)
- Schichten: ui -> application -> domain; infrastructure implementiert Ports
- Geschäftsregeln NUR in domain (Album, Photo). Nie in ui oder infrastructure.
- Keine neuen Dependencies ohne ADR.

## Verboten
- Direkte Commits auf main; Änderungen an .github/workflows ohne Rückfrage
- Löschen oder Abschwächen bestehender Tests
- Secrets, absolute Pfade, System.exit, Thread.sleep in Produktionscode

## Definition of Done für einen PR
- Anforderungs-ID im Titel, Tests für Normal-/Fehler-/Grenzfall, Doku
aktualisiert
- AI_USAGE.md-Eintrag, wenn der Agent beteiligt war
```

- Wird von Copilot, Claude Code, Cursor u. a. automatisch gelesen (AGENTS.md, CLAUDE.md, .github/copilot-instructions.md)
- Versioniert wie Code – Änderungen per PR
- Enthält, was ein neues Teammitglied am ersten Tag wissen müsste
- Regeln aus Block 3 (Schichten) werden hier durchgesetzt
- „Verboten“ ist die Allow-List-Idee aus Block 2/3

Prompts, Skills und Modelle versionieren

Was	Wo im Repository	Warum versionieren?
Prompt-Vorlagen	prompts/describe-photo.md	Ausgabe hängt vom Wortlaut ab; Änderung = Verhaltensänderung = Review
Skills	skills/grill-me/SKILL.md, skills/test-from-ac/SKILL.md	Wiederverwendbar, teilbar, testbar mit Beispielen
Modellkennung + Parameter	config/ai.yml: model: claude-sonnet-4-5, temperature: 0.2	Anbieter ändern Modelle; ohne Kennung ist kein Ergebnis reproduzierbar
Tool-Konfiguration (MCP)	mcp.json: erlaubte Tools, Rechte, Timeouts	Allow-List ist Sicherheitsentscheidung – Änderungen brauchen Review
Evaluation Set	eval/describe-photo/cases.yml (Block 5)	Regressionstest für Prompt- und Modelländerungen
Änderungsgrund	Commit-Message + CHANGELOG	„prompt: shorter descriptions (eval pass rate 72 % → 88 %)“

Regel: Alles, was das Verhalten des Systems beeinflusst, liegt im Repository und geht durch Review. Prompts sind Code.

Übung: AGENTS.md für Ihr Repository

KI-Perspektive

Ein Coding Agent soll morgen den Use Case „Foto entfernen“ implementieren. Was muss er wissen, was darf er nicht?

8 min

Team

AUFTRAG

1. Übernehmen Sie die Vorlage und passen Sie Build-Befehle, Paketnamen und Architekturregeln an Ihr Projekt an.
2. Ergänzen Sie drei projektspezifische Verbote (z. B. „nie Photo.hash() aus dem Dateinamen ableiten).
3. Legen Sie fest, wie ein Agent-PR gekennzeichnet wird (Label „ai-assisted“) und was in AI_USAGE.md stehen muss.

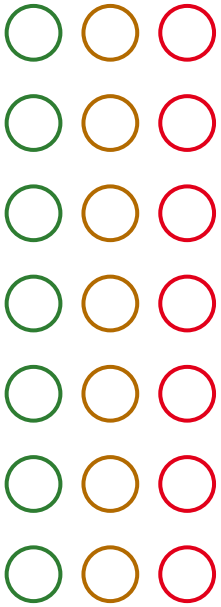
ERGEBNIS AGENTS.md im Repository (Commit „docs: add AGENTS.md“).

DANACH IM PLENUM Ein Team liest seine drei Verbote vor.

Projektcheckpoint Block 4

Teamarbeit

Jedes Team bewertet sich selbst: grün = im Repository sichtbar · gelb = begonnen · rot = fehlt. Für jeden gelben/roten Punkt entsteht ein Issue mit Verantwortlichem.



- Build läuft aus frischem Clone mit einem Befehl (Wrapper vorhanden, Java 21 festgelegt)
- Repository-Struktur nach Schichten; .gitignore; README mit Build/Run-Anleitung
- Vertikaler Schnitt „Album erstellen + anzeigen“ auf main lauffähig
- Mindestens ein PR mit dokumentiertem Review (Ebene 1–3) und Antworten
- CI-Pipeline grün; Branch Protection auf main aktiv
- Commit-Konvention und PR-Template im Repository; Historie ohne „fix“/„WIP“
- AGENTS.md vorhanden; AI_USAGE.md Fall 3 dokumentiert

Abgabe des Zwischenstands: Commit/Tag im Repository bis zum Vorabend des nächsten Blocks.

Bis zum nächsten Block

Bis zum nächsten Block (Teamarbeit)

- Pflichtfunktionalität fertig implementieren: Fotos hinzufügen/anzeigen/entfernen, Titel/Beschreibung, Album umbenennen/löschen – jeweils Issue → Branch → PR → Review → CI (Tag „block-4“)
- Alle Geschäftsregeln aus REQUIREMENTS.md in der Domäne umgesetzt und mit mindestens einem Test je Regel abgesichert
- Release v0.1.0 taggen mit CHANGELOG.md; bekannte Einschränkungen notieren
- Vorbereitung Block 5: Ihre Akzeptanzkriterien mitbringen – sie werden zu Testfällen. JUnit-5-Grundlagen (assertThrows, @ParameterizedTest) auffrischen
- Welche KI-Ausgabe haben Sie heute verworfen, und woran haben Sie den Fehler erkannt?
- Optional: Dependabot aktivieren (.github/dependabot.yml)

Quellen und Materialbasis Block 4

- Chacon, S. & Straub, B.: Pro Git (git-scm.com/book) – Branching, Rebasing, Workflows
- GitHub Docs: GitHub Flow, About pull requests, Building and testing Java with Maven, Copilot coding agent, Repository custom instructions
- Apache Maven / Gradle Dokumentation; JaCoCo Maven Plugin; JUnit 5 User Guide
- Google Engineering Practices: Code Review Developer Guide; conventionalcommits.org; semver.org; keepachangelog.com
- Zeller, A.: Why Programs Fail – Scientific Debugging
- Kim, Humble, Debois, Willis: The DevOps Handbook; OWASP DevSecOps Guideline; OWASP Dependency-Check
- Anthropic: Claude Code Best Practices; agents.md